Technical Documentation for the RATER Competition

Paris, May 5, 2024

This is the technical documentation of the team named *'' Kkiller''* for *RATER Challenge*, hosted by **The Learning Agency LAB**. The competition focused on creating more efficient versions of the algorithms developed in the past Feedback Prize competition series. The task will be to develop an efficient "super-algorithm" that can Segment an essay into individual argumentative elements and evaluate the effectiveness of each discourse element. This task involves Natural Language Processing (NLP) and more precisely Token Classification.

The competition lasts from early February 2024 to end of April 2024 and involve a panel of pre-selected five (5) persons, based on backward performance on early competition organized by the host and as well on personal details (from competitor's resumé).

This competition was great, and we do thank the host for organizing such an event.

0. General information

0.1. The team

Team Name: Kkiller

Number of Team Members: One (01)

Public Leaderboard Score: 0.6523

Public Leaderboard Place: First (01)

Final submission file on the server: /workspace/documents/rater/data/rater/data/kkiller_final_submission.csv

0.2. Team members

Name: Kossi N.

Location: Paris, France

Email: kossy.pro@gmail.com

The team is made up of one person: Kossi N., Kossi is based in Paris and has worked as a Data Scientist Consultant for several companies. He's graduated from the French National School of Statistics and Economic Administration (ENSAE) in Applied Mathematics & Economics. He's also a graduate of the Moroccan National Institute of Statistics and Applied Economy (INSEA) in Statistics, Finance and Actuary. Kossi has grew up in early 2000's in Togo, where he got his bachelor's degree in mathematics and physics.

0.3. My journey during this competition

I've started working on this competition in the very beginning. Thanks to my recent missions and Kaggle competitions in NLP, I was somehow familiar to several textual data processing tasks. Having participated in most, if not all the latest Learning Agency Lab competitions, I was already familiar with the given task. Moreover, I already had some old resource (python scripts and model weights) that I was able to quickly leverage to build a baseline submission. This baseline submission surprisingly scored as high as 0.6622. After several days of development, I was not able to beat this strong baseline model so I decided to reproduce it will cleaner code. In fact, this baseline contains old weights from my teammates who had their own scripts, so combined submission was a bit messy.

I will just recall that for my baseline submission, no prediction was done for the effectiveness targets, using constant values (0.99 for *score_discourse_effectiveness_0* and 0.01 for *score_discourse_effectiveness_1*). During all the competition, it would be very difficult to build a consistent model for the effectiveness targets and given the good scores that can be pulled out by just enhancing the segmentation parts, I decided not to spend too much on this.

Also, my baseline was made up of variants of the popular Microsoft *Deberta* architecture. All my later work has also been based on this same architecture. *Deberta v1* worked out of the box, no need to tweak the tokenizer. On the other side, *Deberta v3* needs some tweaks since its tokenizer does not include some special characters such as: *\n*, *\t* … This stop characters proved to be very important for the segmentation task hence I just naturally add them to v3's tokenizer. My final clean submission manages to score 0.6523. This is currently better than the baseline since the baseline consists of a blend of 13 model weights while my final submission contains just 2 weights.

1. Hardware requirements

All the experiments have been conducted in python version 3.10.12. We train the model mainly on Google Colab (subscription + on the fly compute units acquisition). On Google Colab, one can (randomly) access A100 (40 Gb), V100 (16 Gb) or L4 (24 Gb) GPUs. Depending on the GPU, the batch size and the sequence *maxlen* could range from (2, 768) to (6, 1024) with training time ranging from 15 minutes to 1 hour per epoch. During the last week of the competition, we also use the Learning Agency Lab provided GPU servers (50 Gb RTX 6000) in order to make our results more reproducible.

2. Inference: making prediction with our models

All the scripts needed to make a prediction is in *scripts/inference_script.py*. The *predict()* function is surely what you need. For inference, just run the commands in ***readme.md*. Paths are either absolute or relative to the project root folder**. For a fast demo runs, set *IS_DEBUG* to *True* and just set it to *False* for inference on the whole test set.

Don't forget specifying the right path to your active model path: this is done in the *get_params()* function. Normally, all this should work out of the box.

The prediction is saved under the *TEST_ROOT.parent / submission.csv* path, which defaults to *data/rater/data/submission.csv*.

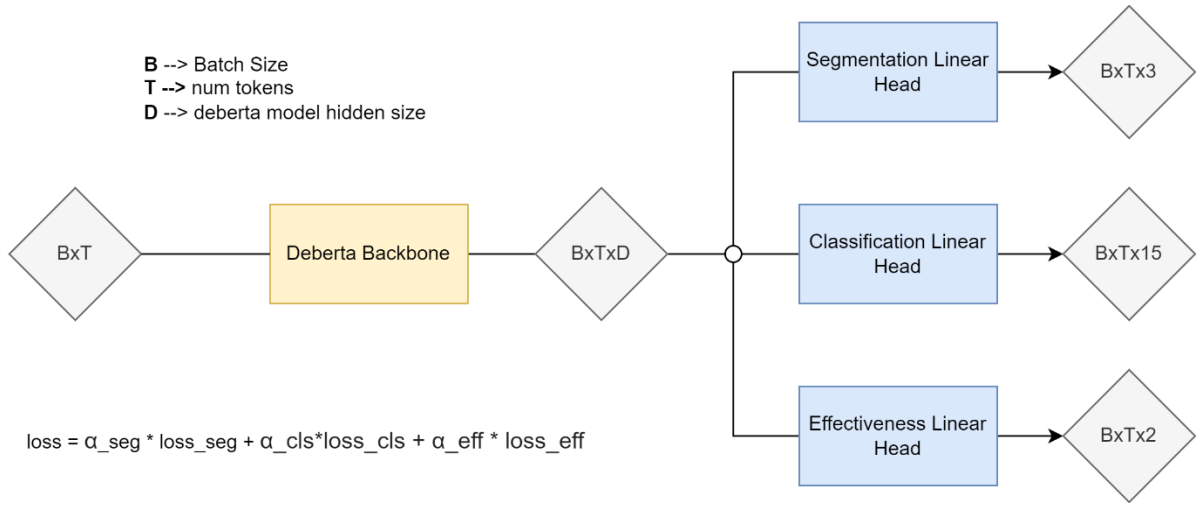
If you have any issue with Python, please follow our indications in readme.md to activate the right environment or to install a Python 3.10.12 from scratch.
Inference can also be conducted in half precision (set *fp16* to *True* in *predict()*) but half precision results are a little bit lower than full precision, so we just stick with the later in our final submission.

3. Training

Training is solely based on the provided data, no external data were used. However, for *deberta-v1-large* we initiate the model weights with old weights from Feedback Prize 2021. These old weights are the ones we used in our 5'th place solution. The old weights was publicly shared and can be found here. The deberta-v3-large model was trained from scratch since no old weights were available to use. Given the good performance of the v3 models (after tokenizer tweaking), weight initialization should have limited impact.

Our models are designed to predict three items (see diagram bellow):

- The token segmentation mask
- The token level class
- The token level effectiveness score



Hence, each of our models is a three-headed model where each head is responsible for one task among the three mentioned above. We apply a cross-entropy loss (eventually with dynamic class weights) to each task and finally take a weighted sum as our final loss. Not only this modeling approach obviously account for all the aspects of this competition but also has the advantage of being fully trainable (as opposed to multi-step training approach), hence we're sure to avoid the usual error propagation risk associated with multi step approaches.

It's also worth to be mentioned that no real post-processing has been done on raw outputs (except for effectiveness scores), indicating that there still room for improvements since by the old Feedback Prize competition, gradient boosting post-processing has been proved to be very effective.

Finally, let recall that we use many techniques for faster and effective model convergence:

- custom scheduling
- warm up
- gradient clipping
- multi-sample dropout
- token masking
- tokenizer augmentation by adding new tokens like <\n > & <\t >
- best weight checkpointing based on eval metrics
- mixed precision training
- weight decaying